Lynx: A Programmatic SAT Solver for the RNA-Folding Problem

Vijay Ganesh[1], Charles W. O'Donnell[1], Mate Soos[2], Srinivas Devadas[1], Martin C. Rinard[1], and Armando Solar-Lezama[1]

[1] Massachusetts Institute of Technology
{vganesh,cwo,devadas,rinard,asolar}@csail.mit.edu
[2] Security Research Labs
mate@srlabs.de

Abstract. This paper introduces Lynx, an incremental programmatic SAT solver that allows non-expert users to introduce domain-specific code into modern conflict-driven clause-learning (CDCL) SAT solvers, thus enabling users to guide the behavior of the solver.

The key idea of Lynx is a *callback interface* that enables non-expert users to *specialize* the SAT solver to a class of Boolean instances. The user writes specialized code for a class of Boolean formulas, which is periodically called by Lynx's search routine in its inner loop through the callback interface. The user-provided code is allowed to examine partial solutions generated by the solver during its search, and to respond by adding CNF clauses back to the solver dynamically and incrementally. Thus, the user-provided code can specialize and influence the solver's search in a highly targeted fashion. While the power of incremental SAT solvers has been amply demonstrated in the SAT literature and in the context of DPLL(T), it has not been previously made available as a programmatic API that is easy to use for non-expert users. Lynx's callback interface is a simple yet very effective strategy that addresses this need.

We demonstrate the benefits of Lynx through a case-study from computational biology, namely, the RNA secondary structure prediction problem. The constraints that make up this problem fall into two categories: structural constraints, which describe properties of the biological structure of the solution, and energetic constraints, which encode quantitative requirements that the solution must satisfy. We show that by introducing structural constraints on-demand through user provided code we can achieve, in comparison with standard SAT approaches, upto 30x reduction in memory usage and upto 100x reduction in time.

1 Introduction

Conflict-driven clause-learning (CDCL) Boolean SAT solvers have had a huge impact on a variety of domains ranging from program analysis to AI [3]. This success can partly be attributed to their simple interface and powerful heuristics. In many cases, a straightforward translation from a program analysis or AI problem into Boolean formulas in CNF (conjunctive normal form) format is sufficient to leverage the power of the solver. Unfortunately, there are many other important domains (e.g., biology) where straightforward translation of problems to CNF clauses leads to formulas that are too

A. Cimatti and R. Sebastiani (Eds.): SAT 2012, LNCS 7317, pp. 143–156, 2012.

large or complex for solvers to handle. For many of these domains, however, small domain-specific modifications to the solver can make SAT-based solution feasible. The challenge addressed by this paper is to enable users to make these small adaptations with minimal effort and without breaking subtle invariants in the solver implementation. The solution we provide allows for the resultant specialized solver to be adaptive, efficient for the problem-at-hand, and easy to build and maintain. Equally important, users are not burdened with knowing too much about the internals of SAT solvers and related technologies.

1.1 Our Contributions

- To address the problem described above, we created the solver Lynx that extends CryptoMiniSat [23] with an API allowing user-provided code to examine partial solutions generated by the SAT solver and add CNF clauses back to the solver in response. The added code is called inside the inner loop of the SAT solver, allowing the user to tightly integrate problem-specific clause-generation heuristics into the solver.

  We call solvers extended in this way *programmatic*, i.e., the user can programmatically influence solver behavior and adapt it to their specific problem domain in ways that are difficult to achieve otherwise. Programmatic solvers address the "solvers are unpredictable black boxes" problem by giving users more control over their search heuristics.
- Using Lynx we developed the first SAT based tool for solving the RNA-folding prediction problem. We present a detailed experimental evaluation of our technique in comparison with standard approaches. We use the above-mentioned callback interface in efficiently translating the RNA prediction problem into Boolean formulas. The interface allows Lynx to incrementally translate the RNA-folding structure inside the inner loop of the SAT solver, allowing a tighter, highly targeted and more efficient integration of the SAT solver and the translator.

1.2 Existing Approaches to Incremental and Adaptive Solving

Incremental solvers, that use some form of *abstraction-refinement* [3], have been proposed as a solution to the above-mentioned issue of *simple but inefficient translations* from problems to Boolean formulas. Instead of translating the entire input problem-instance into a potentially very large Boolean formula in one step, *abstraction-refinement approaches* translate the input instance into Boolean formulas incrementally and call the solver on these incrementally generated formulas. Such formulas are abstractions of the input instance and are often easier to solve than the entire input instance. The solver terminates if it gets the correct result to the input instance by solving an abstraction. Otherwise the solver iteratively refines the abstractions as necessary until it gets the correct result. Typically these abstractions and their refinements are performed by a layer outside the inner loop of the SAT solver. For an excellent reference on abstraction-refinement strategies refer to the Handbook of Satisfiability [3].

Such incremental SAT solvers with an outside abstraction-refinement loop are relatively easy to build. However, the problem with such an approach is that it may not be